Lab Report: Lab 5 - Task Management Module Implementation

Kyte

Contents

1. Cover Page
2. Abstract/Summary
3. Lab Specific Section: V. Task Management Module 3.1 Objectives 3.2 Technology and Tools 3.3 Data Model 3.4 Service API Endpoints 3.5 Testing and Verification
4. Conclusion & Reflection

1. Cover Page

Title: Task Management Module Implementation for the Kyte Project Management Tool

Course Info: Software Architecture

Student Details:

Họ và Tên	MSSV
Trần Thành Long	23010070
Nguyễn Xuân Mạnh	23010045

Date: Dec 15, 2025

2. Abstract/Summary

This lab implements the Task Management module in Kyte. The module owns task data, exposes REST endpoints, and supports search, detail, and move operations in the Kanban workflow.

3. Lab Specific Section: V. Task Management Module

3.1 Objectives

- Build the `tasks` module inside the monorepo structure.
- Define schema and persistence with Drizzle.
- Expose service contracts for task CRUD and movement.
- Verify the module independently before full integration.

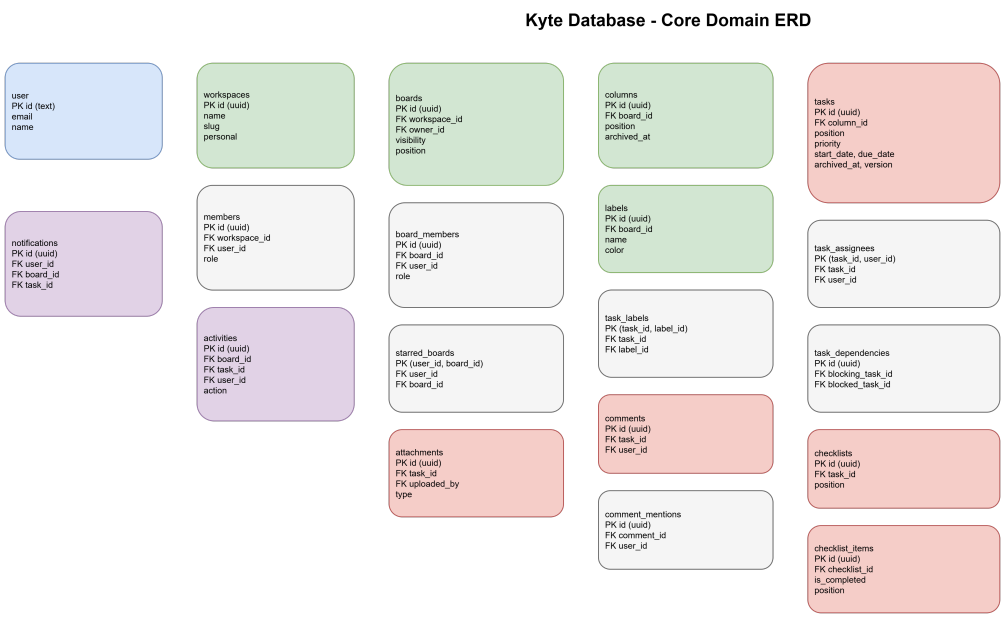
3.2 Technology and Tools

Tool	Purpose
Bun	Runtime for TypeScript services.
Elysia	Presentation layer for task APIs.
Drizzle ORM	Typed database access.
PostgreSQL	Persistent storage for tasks.
cURL / Postman	Validate API behavior.

3.3 Data Model

Core task fields used in Kyte:

Field	Type	Purpose
`id`	uuid	Primary key
`title`	text	Task title
`description`	text	Task description
`position`	text	Fractional index for ordering
`columnId`	uuid	Column reference
`archivedAt`	timestamp	Soft delete marker
`createdAt`	timestamp	Created time



Data Model

3.4 Service API Endpoints

Endpoint	Method	Description
`/v1/tasks/column/:columnId`	GET	List tasks in a specific column
`/v1/tasks/:id`	GET	Fetch task details
`/v1/tasks`	POST	Create task in a column (via `columnId` in body)
`/v1/tasks/:id`	PATCH	Update task fields
`/v1/tasks/:id/move`	PATCH	Move task to a new column or position

3.5 Testing and Verification

Minimal checks using cURL:

1. Column tasks

```
curl -X GET http://localhost:3000/v1/tasks/column/your_column_id
```

2. Fetch task by id

```
curl -X GET http://localhost:3000/v1/tasks/your_task_id
```

3. Create task

```
curl -X POST http://localhost:3000/v1/tasks \
  -H "Content-Type: application/json" \
                                    -d                  '{"title":"Sample
        Task","columnId":"your_column_id","position":"a0"}'
```

4. Move task

    - Send `PATCH /v1/tasks/:id/move` with `columnId`, `position`, and `version` in the body.

4. Conclusion & Reflection

The task module is now a self contained unit with clear contracts, typed persistence, and reliable real time integration points. This enables the frontend to consume task data consistently and supports future extensions such as automation and analytics.